

Calculadora Acumulador — Processador RISC-V

RV32I na DE10-Lite

Autor: João Moreti · Arquitetura e Organização de Computadores · PUCPR 2026/1
Guia de uso da placa, dos testes e dos pontos de defesa

1. O que é esta calculadora

A calculadora é um **programa em assembly RISC-V** (`asm/calculator.s`) rodando no próprio processador construído em VHDL — não é um circuito dedicado. É isso que *prova* que o processador funciona de verdade: ele busca instruções na ROM, lê as chaves, calcula na ULA, guarda no banco de registradores e escreve no display.

É uma calculadora **acumulador**: você entra um valor de cada vez e aperta um botão para aplicá-lo ao resultado acumulado (`acc = acc <op> valor`).

2. O que cada switch faz

SW9 SW8 | SW7 SW6 SW5 SW4 SW3 SW2 SW1 SW0

+ - op --+ +----- valor (0..255) -----+

| 128 64 32 16 8 4 2 1 <- peso de cada chave

SW0–SW7 — o VALOR (número binário de 0 a 255)

Cada chave vale uma potência de 2; você "monta" o número em binário (chave para cima = 1, para baixo = 0):

Para entrar...	Some os pesos	Chaves para cima	Binário SW[7:0]
10	8 + 2	SW3, SW1	00001010
20	16 + 4	SW4, SW2	00010100
100	64 + 32 + 4	SW6, SW5, SW2	01100100
255	todos	SW0–SW7	11111111

SW8, SW9 — a OPERAÇÃO

SW9	SW8	Código	Operação
baixo	baixo	00	+ (soma)
baixo	cima	01	- (subtração)
cima	baixo	10	& (AND bit a bit)
cima	cima	11	(OR bit a bit)

Botões e saídas

Elemento	Função
KEY1	Enter / = — aplica a operação ao acumulador (uma vez por aperto)
KEY0	Limpar — zera o acumulador (reset do processador)
HEX0–HEX5	Acumulador em decimal com sinal
LEDR8	Acende quando o resultado é negativo
LEDR9	Overflow (não acende na faixa normal)
LEDR0, LEDR1	Espelham a operação selecionada (SW8, SW9)

3. Como usar — exemplo: 10 + 20 – 5 = 25

- 1. KEY0 → zera (display mostra 0)
- 2. SW = 00001010 (10), op 00 → KEY1 → display 10
- 3. SW = 00010100 (20), op 00 → KEY1 → display 30
- 4. SW = 00000101 (5), op 01 → KEY1 → display 25

4. Os testes (o que o testbench prova)

O `sim/tb_calc.vhd` simula essa interação — reset e 5 pressões encadeadas — conferindo o acumulador a cada passo. Validado no GHDL: **5/5 testbenches ALL TESTS PASSED**.

#	Valor	Op	Acumulador	O que prova
1	10	+	10	soma e acumulação inicial
2	20	+	30	acumulação (usa o valor anterior)
3	5	–	25	subtração
4	24	&	24	AND (25 & 24)
5	1		25	OR (24 1)

Os outros testbenches validam as peças isoladas: `tb_alu` (ULA), `tb_regfile` (banco de registradores), `tb_bin2bcd` (conversão binário→decimal) e `tb_core` (CPU completa rodando um programa).

5. Os "segredos" bons para a defesa

1. Multiplexação no tempo (a sacada central). A placa só tem 10 switches. Em vez de exigir os dois operandos ao mesmo tempo (o que limitaria a somas ≤ 62), cada valor é entrado em *momentos diferentes* e guardado num registrador. Isso desacopla "quantos bits consigo entrar" de "quantos switches existem" — é o que um computador de verdade faz: reusa recursos no tempo.

2. Debounce por software + detecção de borda. O laço lê a I/O a ~3 MHz; um aperto físico dura dezenas de ms e o contato "treme" (bounce). Sem tratamento, 1 aperto viraria milhares de operações. O programa detecta a pressão → espera (`delay ~3 ms`) → reconfirma → aplica **uma vez** → espera soltar. Garante **1 aperto = 1 operação**.

3. Registradores do banco não têm reset. O reset zera o PC (volta ao início), mas **não** zera os registradores. Por isso o programa precisa fazer `addi s0, x0, 0` à mão para iniciar o acumulador em zero. Só o `x0` é garantidamente zero por hardware.

4. A calculadora é um PROGRAMA, não um circuito dedicado. Usa instruções RV32I reais: `lw / sw` para falar com a I/O, `beq / bne` para decodificar a operação e `jal / ret` para a sub-rotina de atraso. É a prova de que o processador funciona.

5. Entra binário, sai decimal. Um bloco `bin2bcd` por *double-dabble* (shift-and-add-3) converte o acumulador para BCD, e cada dígito vira segmentos no display. Você digita binário nas chaves e lê decimal — a conversão é feita em hardware.

6. I/O mapeada em memória. Chaves, botões, LEDs e display são **endereços de memória**: `0x2000` =chaves, `0x2004` =botões, `0x2010` =LEDs, `0x2020` =display. Para o processador, ler uma chave é um `lw` comum — não existe instrução especial de I/O.

7. Botões são ativo-baixo. Os KEY da DE10-Lite dão `0` quando pressionados; o hardware inverte (`not KEY`) antes de entregar ao programa. Por isso `rst <= not KEY(0)` .

6. Resumo do mapa de I/O (base 0x2000)

Endereço	R/W	Conteúdo
<code>0x2000</code>	R	SW[9:0] (valor + operação)
<code>0x2004</code>	R	bit0 = KEY0, bit1 = KEY1 (1 = pressionado)
<code>0x2010</code>	W	LEDR[9:0]
<code>0x2020</code>	W	valor exibido (decimal com sinal nos HEX)